

Multilayer Perceptron Formulae

Pei-Yuan Sun, Allan Cosmo

July 28, 2025

$$Z = \sum_{i=1}^n (x_i \cdot w_i) + b_i$$

Z is the linear combination and is not activated. x_i is your input (either a vector or an array).

w_i is the weight for each input. b_i is the bias for each neuron node.

$$A = f(Z)$$

A is activated. $f(\cdot)$ is activation function.

Activation functions:

Tanh:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

ReLU:

$$f(z) = \max(0, z)$$

Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Softmax:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}}$$

Forward propagation

- Calculating Z (linear combination of inputs)
- Applying activation function $A = f(Z)$
- Repeating layer-by-layer until final prediction $\hat{y}$

In layer L :

Weight: $W^{(L)}$, Biases: $b^{(L)}$, Non-activated combination: $Z^{(L)}$,

Activated output: $A^{(L)}$, Activation function: $f^{(L)}(\cdot)$.

Input:

$$x^{(L_{in})}$$

Output:

$$Z^{(L_{out})} = A^{(L_{hidden})} \times w^{(L_{out})} + b^{(L_{out})}$$

$$\hat{y} = f^{(L_{out})}(Z^{(L_{out})})$$

$\hat{y}$ is model predict output. Loss function:

Mean Squared Error, MSE

$$L(y, \hat{y}) = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

Binary Cross-Entropy (for sigmoid output):

$$L(y, \hat{y}) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Categorical Cross-Entropy (for softmax output):

$$L(y, \hat{y}) = -\sum_{i=1}^m (y_i \times \log(\hat{y}_i))$$

Backward propagation

Output Gradient

$$\delta^{(L_{out})} = \frac{\partial L}{\partial A^{(L_{out})}} \times f^{(L_{out})}(Z^{(L_{out})})$$

$$\frac{\partial L}{\partial W^{(L_{out})}} = A^{(L_{hidden})T} \times \delta^{(L_{out})}$$

$$\frac{\partial L}{\partial b^{(L_{out})}} = \text{sum}(\delta^{(L_{out})}, \text{axis} = 0)$$

Hidden Gradient

$$\delta^{(L-1)} = \delta^{(L)} \times (W^{(L)})^T \times f^{(L-1)}(Z^{(L-1)})$$

$$\frac{\partial L}{\partial W^{(L-1)}} = A^{(L-1)T} \times \delta^{(L-1)}$$

$$\frac{\partial L}{\partial b^{(L-1)}} = \text{sum}(\delta^{(L-1)}, \text{axis} = 0)$$

Update weights and biases(Gradient Descent)

$$W_{new}^{(L)} = W_{old}^{(L)} - \text{learning_rate} \times \frac{\partial L}{\partial W^{(L)}}$$

$$b_{new}^{(L)} = b_{old}^{(L)} - \text{learning_rate} \times \frac{\partial L}{\partial b^{(L)}}$$